

Bioinformatics command cheatsheet

1 • Anatomy of a command	2
2 • Conda workflow	3
3 • Bioinformatics essentials	4
4 • PyMOL quick reference	4

Quick reference · Ubuntu/WSL and macOS Terminal for shell examples and the conda workshop environment. On Windows, work inside WSL Ubuntu: the environment's bioconda tools do not run in the native Miniforge Prompt. Run PyMOL commands inside the PyMOL prompt, not the system terminal. Substitute your own filenames; examples are not a validated analysis pipeline.

1 • Anatomy of a command

```
program subcommand --option value -f input.ext > output.ext
```

| | | | | | | | | |
| | | | | | | | | shell writes stdout to file
| | | | | | | | positional input filename
| | | | | | short flag (tool-specific)
| | | | option value
| | long option (tool-specific)
| action (subcommand)
executable

Real example — run at the **system terminal** after `conda activate protein modeling`:

```
pymol -cq -d "fab ACDE, peptide; save peptide.pdb, peptide; quit"
├─ pymol: executable
├─ -c: no GUI; -q: quiet startup
├─ -d: execute the quoted commands
├─   └─ fab: build a peptide
├─       └─ ACDE: toy amino-acid sequence
├─           └─ peptide: name of the object
├─               └─ save: write peptide.pdb from the object
└─                   └─ quit: exit PyMOL
```

The quotes keep the PyMOL commands together as one argument; semicolons separate `fab`, `save`, and `quit`. In `save peptide.pdb`, `peptide`, the final `peptide` names the object to export. The file lands in the current folder. This command has **no subcommand or long option**: not every program uses every part of the generic diagram. `fab ACDE` produces a toy model, not experimental data.

Shell shorthand	Meaning	Example
~ · . · . .	Home · current · parent directory	cd ~/workshop
" . . . " · *	Quote paths with spaces · match filenames	ls "my data" · ls *.fastq.gz
> · >> · 2>	Overwrite stdout · append stdout · save stderr	seqkit stats reads.fastq.gz > stats.txt
	Pipe stdout to another command	zcat reads.fastq.gz less

Everyday shell

Navigate and inspect	Files and help
<code>pwd</code> — current location	<code>ls -lh</code> — list files/sizes
<code>cd ~/workshop</code> — enter folder	<code>mkdir -p results</code> — make folder
<code>less sequences.fasta</code> — read text (q to exit)	<code>cp input.fasta results/</code> — copy
<code>grep 'BRCA1' annotations.tsv</code> — search	<code>mv old.fasta new.fasta</code> — rename
<code>wc -l samples.txt</code> — count lines	<code>samtools view --help</code> — show options

More Linux essentials

Standard Unix commands you reach for often, beyond the basics above. All work in Ubuntu/WSL and macOS Terminal. Sequence files are large, so **peek** at them rather than opening them whole.

Peek inside files	Filter, sort, and find
<code>head -n 20 annotations.tsv</code> — first lines	<code>sort samples.txt</code> — sort lines
<code>tail -n 20 run.log</code> — last lines	<code>sort ids.txt</code> <code>uniq -c</code> — count each unique line

Peek inside files	Filter, sort, and find
<code>tail -f run.log</code> — follow a growing log (Ctrl+C stops)	<code>cut -f1,2 annotations.tsv</code> — pick TSV columns
<code>cat notes.txt</code> — print a small file	<code>grep -c '>' reference.fasta</code> — count FASTA records
<code>gunzip -c reads.fastq.gz head -n 8</code> — peek inside a .gz	<code>find . -name "*.fasta"</code> — find files by name
Files, folders, permissions	Compress and extract
<code>chmod +x setup_conda_envs.sh</code> — make a script runnable	<code>tar -xzf data.tar.gz</code> — extract a .tar.gz download
<code>./setup_conda_envs.sh</code> — run a script in this folder	<code>tar -czf out.tar.gz folder/</code> — archive a folder
<code>rm -i old.fasta</code> — delete, prompting first	<code>gunzip -k reads.fastq.gz</code> — decompress, keep the .gz
<code>rmdir emptydir</code> — remove an empty folder	<code>gzip -k reads.fastq</code> — compress, keep the original
<code>ln -s target linkname</code> — make a shortcut (symlink)	<code>du -sh results/</code> — total size of a folder
System and help	Windows/WSL and macOS
<code>man ls</code> — full manual (q to quit)	<code>cd /mnt/c/...</code> — reach your Windows C: files (WSL)
<code>which samtools</code> — where a command lives	<code>explorer.exe .</code> — open this folder in Windows (WSL)
<code>df -h .</code> — free disk space here	<code>open .</code> — open this folder in Finder (macOS)
<code>history</code> — list recent commands	<code>nano notes.txt</code> — quick editor (Ctrl+O save, Ctrl+X exit)
<code>clear</code> — clear the screen	<code>~</code> is your fast Linux home; <code>/mnt/c</code> is the Windows drive

Workshop data lives at `/mnt/c/PIPELINE/Ologist_Workshop/Datasets` (that is C:

`\PIPELINE\Ologist_Workshop\Datasets` in Windows). Keep new Linux projects under your home, `~/workshop`, when practical; reach existing Windows files through `/mnt/c/`.

Keyboard	Does
Tab	Auto-complete a file or command name
Up / Down arrow	Reuse the previous / next command
Ctrl+C	Cancel the command that is running
Ctrl+R	Search your command history
Ctrl+L	Clear the screen (same as <code>clear</code>)
Ctrl+A / Ctrl+E	Jump to the start / end of the line
q	Quit <code>less</code> , <code>man</code> , and other pagers

Check `pwd` before file writes or deletion. `rm` can delete without a recycle bin. On Windows, run these inside WSL Ubuntu; the native Miniforge Prompt does not support every Unix shell example here.

2 • Conda workflow

The whole workshop runs in **one** environment, `protein_modeling`, on Linux or macOS (on Windows, inside WSL Ubuntu). It holds Biopython, pandas, PyMOL, the bioinformatics tools, and the docking engine together, so you never switch environments mid-exercise. Build it once with the setup script, then activate it on later visits:

```
conda --version
bash setup_conda_envs.sh      # from the Ologist_Workshop project folder
conda activate protein_modeling
```

The script creates the environment and skips it if it already exists. It runs, in effect, this single command (which you can run directly if you do not have the project folder):

```
conda create -n protein_modeling \
  -c conda-forge -c bioconda python=3.11 \
  biopython pandas pymol-open-source \
  fastqc seqkit bwa samtools bcftools blast \
  vina meeko rdkit
```

Check the install, then leave the environment when you are done:

```
conda info --envs
python -c "import Bio, pandas, pymol; print('biopython', Bio.__version__, 'pandas',
pandas.__version__)"
conda deactivate
```

Task	Command
See packages in active environment	conda list
Install a package into the active environment	conda install -c conda-forge -c bioconda pdbfixer
Export / recreate on a compatible system	conda env export -n protein_modeling > environment.yml conda env create -f environment.yml
Remove an inactive environment (permanent)	conda env remove -n old-env

Avoid `sudo conda` and project packages in `base`. The bioconda tools (samtools, bwa, blast, and the rest) require Linux or macOS, so the environment cannot be built in the native Windows Miniforge Prompt — use WSL Ubuntu there. A full environment export can contain platform-specific builds and a local `prefix`; review before sharing across systems.

3 • Bioinformatics essentials

Run in the system terminal with `protein_modeling` active. Check inputs, read pairing, assembly version, and output files before biological interpretation. `*.fastq.gz` means all matching files in the current folder.

Task	Command
FASTA/FASTQ record and length summary	seqkit stats *.fastq.gz
Read-quality report (make directory first)	mkdir -p results.fastqc -o results reads.fastq.gz
Extract a named FASTA record	seqkit grep -n -p geneA reference.fasta > geneA.fasta
Index a reference for BWA	bwa index reference.fasta
Index sorted BAM / mapping summary	samtools index sample.sorted.bam samtools flagstat sample.sorted.bam
Inspect a genomic interval (indexed BAM)	samtools view sample.sorted.bam chr1:1000-2000
VCF header / variant fields	bcftools view -h variants.vcf.gz bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\n' variants.vcf.gz

Paired-end alignment (after `bwa index`; R1 and R2 must be matching mates):

```
bwa mem reference.fasta R1.fastq.gz R2.fastq.gz |
samtools sort -o sample.sorted.bam -
```

Local nucleotide comparison (not an online NCBI search):

```
blastn -query query.fasta -subject reference.fasta -outfmt 6 > hits.tsv
```

4 • PyMOL quick reference

After activating `protein_modeling`, run `pymol` at the system terminal. Enter the commands below at **PyMOL>**, one at a time. The `fab` peptide is a toy model, not experimental data.

Goal	PyMOL command
Make a tiny peptide / open a local PDB	fab ACDE, peptide.load protein.pdb, protein
Download a PDB entry (internet required)	fetch lubq, async=0
Reset display / show cartoon or sticks	hide everything, protein.show cartoon, protein.show sticks, peptide
Color and focus	color cyan, protein.zoom protein

Goal	PyMOL command
Select chain A, residues 10-20	<code>select site, protein and chain A and resi 10-20</code>
Highlight selection	<code>show sticks, site.color yellow, site</code>
Align a similar structure onto a reference	<code>align mobile, target</code>
Background / rendered image	<code>bg_color white.ray 1200, 900.png figure.png</code>
Save editable session / coordinates	<code>save scene.pse.save protein.pdb, protein</code>
Help / exit	<code>help color.quit</code>

For a headless run without the GUI, use the system-terminal example in Section 1 after activating `protein_modeling`.

`.pse` keeps the editable scene; `.pdb` keeps coordinates. Output goes into the current folder unless you give a path. Cite the structure source when using real PDB entries.

Further help: [setup manual](#) · [protein modeling & docking manual](#) · [conda guide](#) · [PyMOL commands](#).